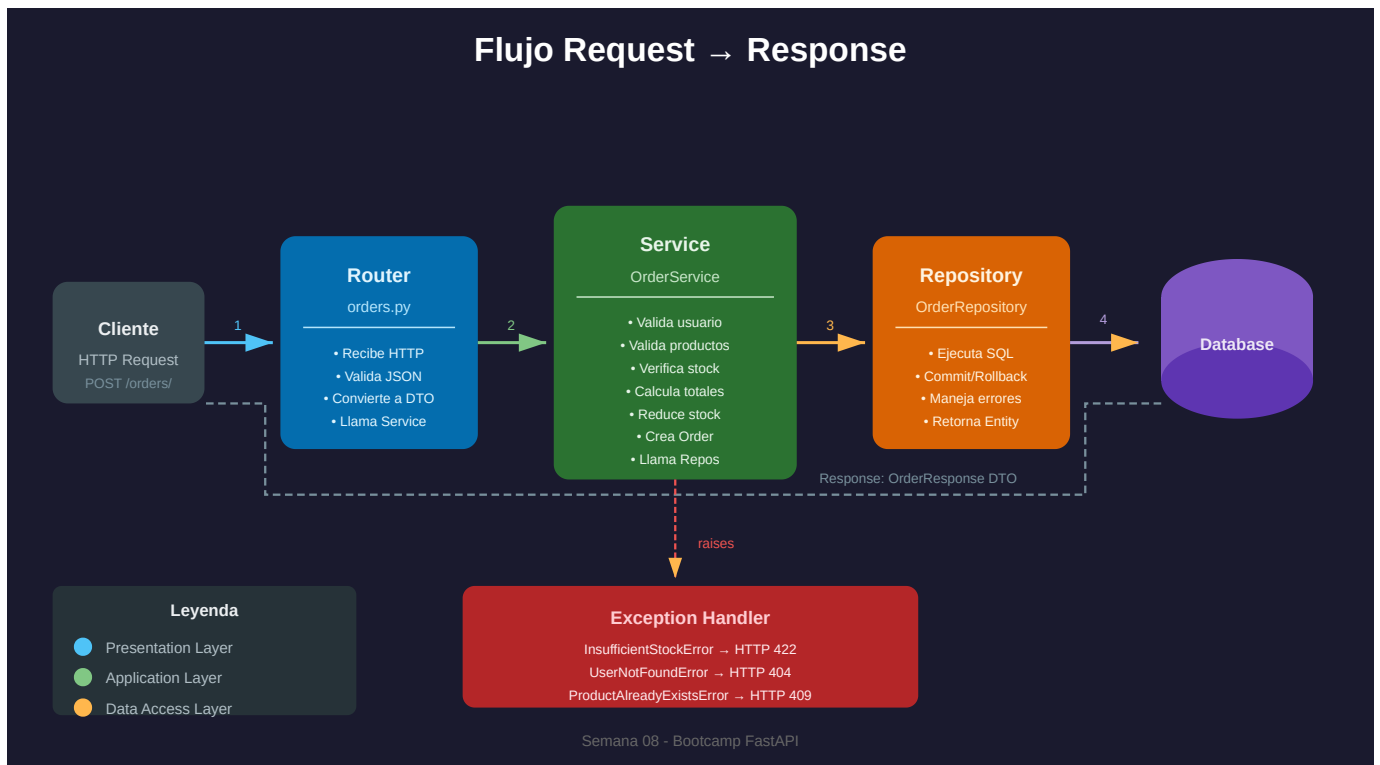


Flujo de Datos Completo

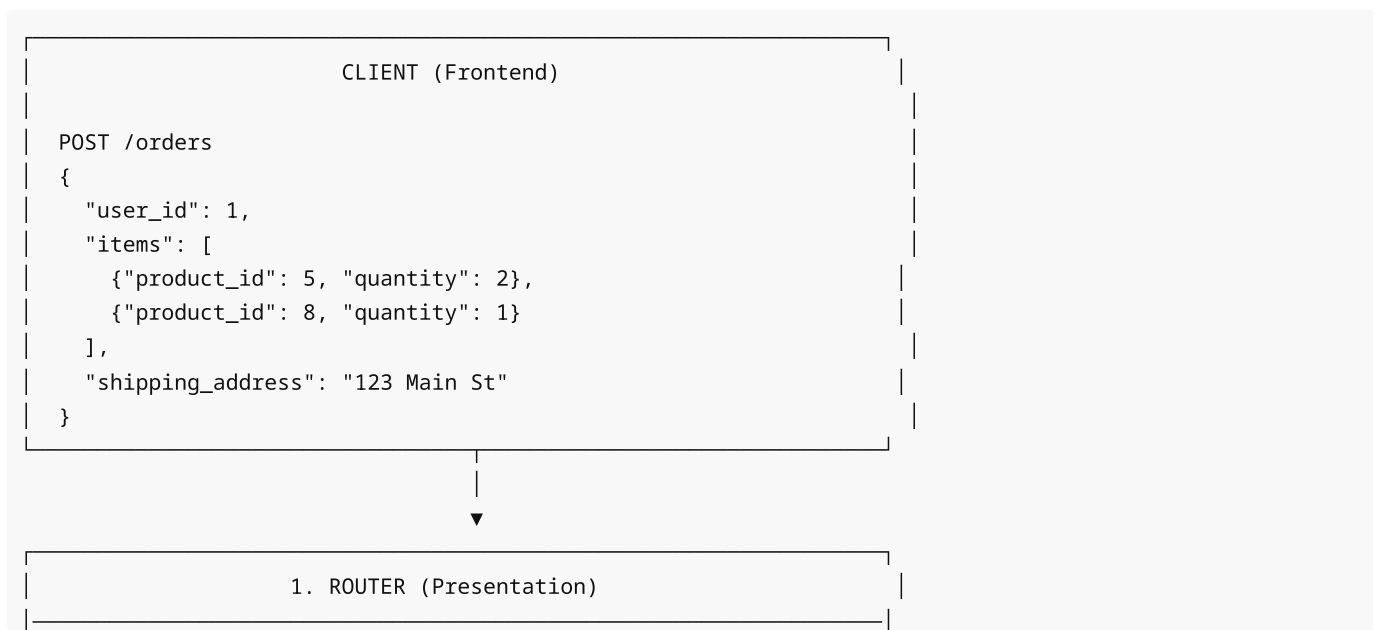


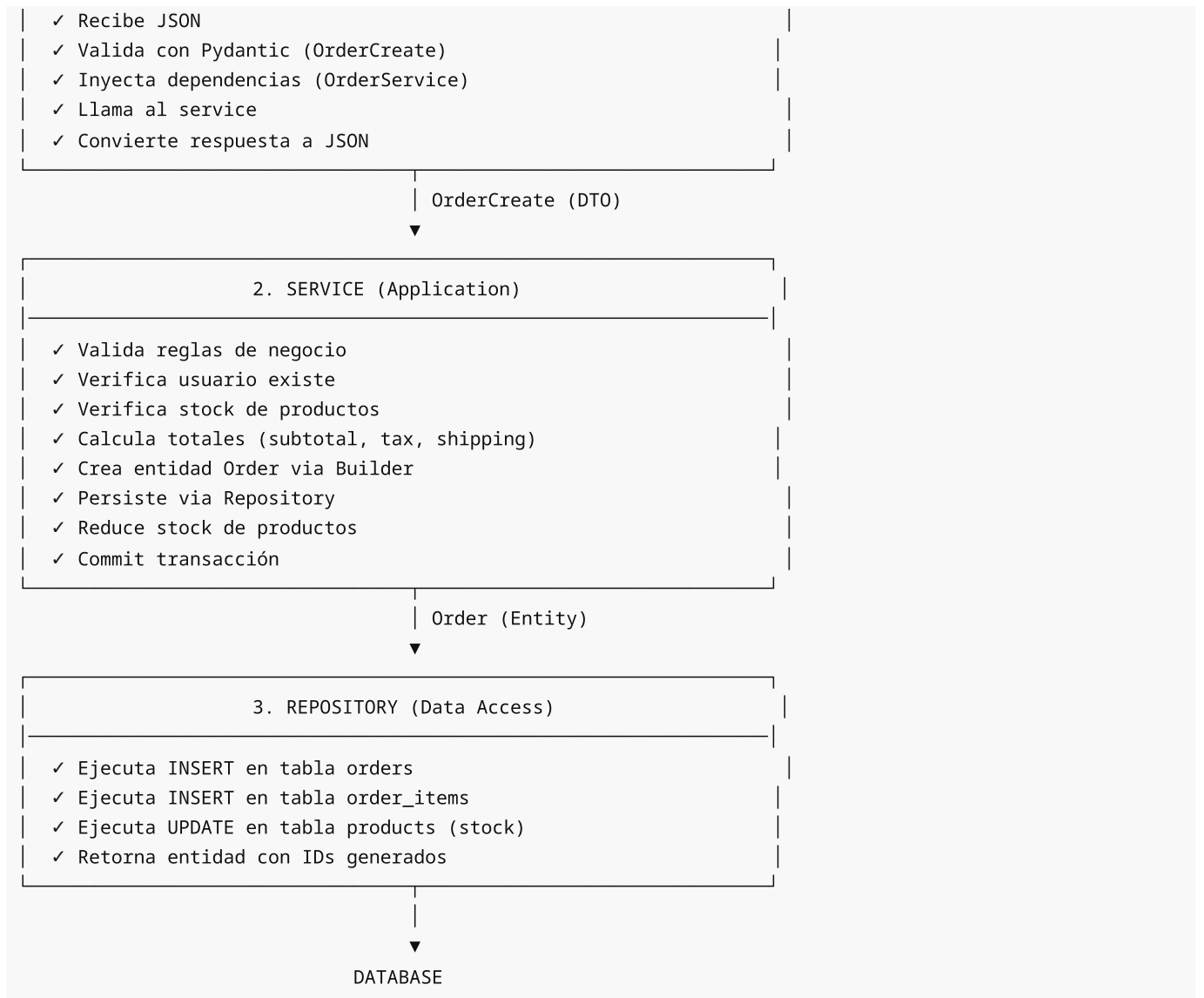
Objetivos

- Entender el flujo completo de una request
- Ver la transformación de datos en cada capa
- Identificar responsabilidades en cada paso
- Implementar un flujo end-to-end

El Viaje de una Request

Veamos el flujo completo de `POST /orders` para crear un pedido:





Implementación Paso a Paso

1. DTOs (Schemas)

```
# schemas/order.py
from pydantic import BaseModel, Field
from datetime import datetime

class OrderItemCreate(BaseModel):
    """Item del pedido - entrada"""
    product_id: int
    quantity: int = Field(..., gt=0)

class OrderCreate(BaseModel):
    """Crear pedido - entrada"""
    user_id: int
    items: list[OrderItemCreate] = Field(..., min_length=1)
    shipping_address: str = Field(..., min_length=10)
    notes: str | None = None
```

```

class OrderItemResponse(BaseModel):
    """Item del pedido - salida"""
    id: int
    product_id: int
    product_name: str
    quantity: int
    unit_price: float
    subtotal: float

class OrderResponse(BaseModel):
    """Pedido - salida"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    user_id: int
    status: str
    items: list[OrderItemResponse]
    subtotal: float
    tax: float
    shipping_cost: float
    total: float
    shipping_address: str
    created_at: datetime

```

2. Router (Presentation Layer)

```

# routers/orders.py
from fastapi import APIRouter, Depends, status

from schemas.order import OrderCreate, OrderResponse
from services.order import OrderService
from dependencies import get_order_service

router = APIRouter(prefix="/orders", tags=["orders"])

@router.post(
    "/",
    response_model=OrderResponse,
    status_code=status.HTTP_201_CREATED
)
def create_order(
    data: OrderCreate,
    service: OrderService = Depends(get_order_service)
):
    """
    Crea un nuevo pedido.

    - Valida datos de entrada (Pydantic)
    - Delega lógica al Service
    - Retorna OrderResponse

```

```
"""
return service.create_order(data)
```

3. Service (Application Layer)

```
# services/order.py
from datetime import datetime

from schemas.order import OrderCreate, OrderResponse
from models.order import Order, OrderItem, OrderStatus
from repositories.order import OrderRepository
from repositories.product import ProductRepository
from repositories.user import UserRepository
from unit_of_work import UnitOfWork
from exceptions.order import InsufficientStockError
from exceptions.user import UserNotFoundError
from exceptions.product import ProductNotFoundError

class OrderService:
    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_order(self, data: OrderCreate) -> Order:
        """
        Crea un pedido aplicando reglas de negocio.

        1. Valida usuario
        2. Valida productos y stock
        3. Construye el pedido
        4. Actualiza inventario
        5. Persiste todo en una transacción
        """
        # 1. Validar usuario existe
        user = self.uow.users.get_by_id(data.user_id)
        if not user:
            raise UserNotFoundError(data.user_id)

        # 2. Validar productos y stock
        order_items = []
        subtotal = 0

        for item_data in data.items:
            product = self.uow.products.get_by_id(item_data.product_id)
            if not product:
                raise ProductNotFoundError(item_data.product_id)

            if product.stock < item_data.quantity:
                raise InsufficientStockError(
                    product_id=product.id,
                    requested=item_data.quantity,
                    available=product.stock
                )

            # Crear item
```

```

        item = OrderItem(
            product_id=product.id,
            quantity=item_data.quantity,
            unit_price=product.price,
            subtotal=product.price * item_data.quantity
        )
        order_items.append(item)
        subtotal += item.subtotal

# 3. Calcular totales
tax = subtotal * 0.16
shipping = 0 if subtotal >= 100 else 10
total = subtotal + tax + shipping

# 4. Crear orden
order = Order(
    user_id=data.user_id,
    status=OrderStatus.PENDING,
    subtotal=subtotal,
    tax=tax,
    shipping_cost=shipping,
    total=total,
    shipping_address=data.shipping_address,
    notes=data.notes,
    items=order_items,
    created_at=datetime.utcnow()
)

# 5. Persistir
saved_order = self.uow.orders.add(order)

# 6. Reducir stock
for item_data in data.items:
    product = self.uow.products.get_by_id(item_data.product_id)
    product.stock -= item_data.quantity
    self.uow.products.update(product)

# 7. Commit transacción
self.uow.commit()

return saved_order

```

4. Repository (Data Access Layer)

```

# repositories/order.py
from sqlalchemy import select
from sqlalchemy.orm import Session, joinedload

from repositories.base import BaseRepository
from models.order import Order

class OrderRepository(BaseRepository[Order]):
    def __init__(self, db: Session):
        super().__init__(db, Order)

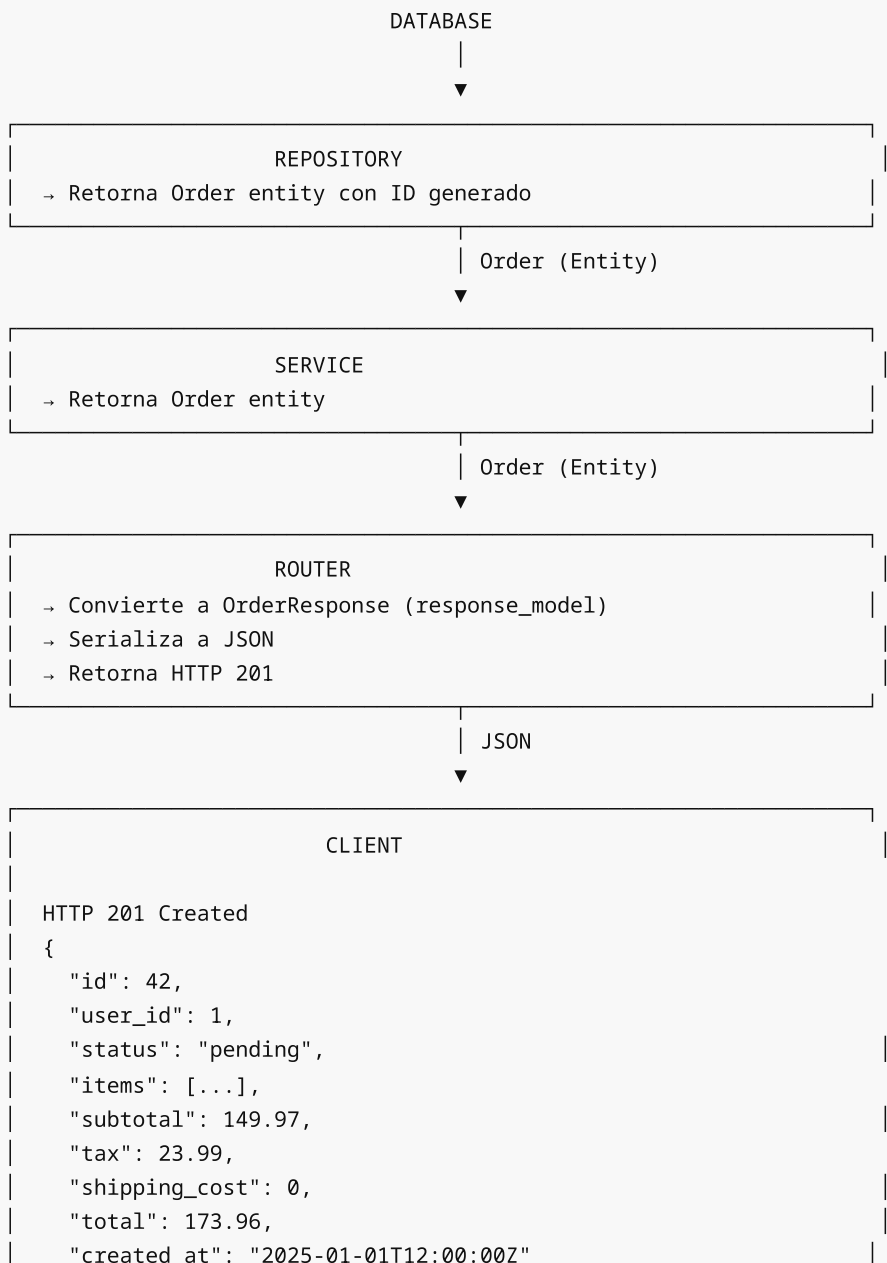
```

```
def get_by_id_with_items(self, order_id: int) -> Order | None:
    """Obtiene orden con items precargados"""
    stmt = (
        select(Order)
        .options(joinedload(Order.items))
        .where(Order.id == order_id)
    )
    return self.db.execute(stmt).unique().scalar_one_or_none()

def get_by_user(self, user_id: int) -> list[Order]:
    """Obtiene órdenes de un usuario"""
    stmt = select(Order).where(Order.user_id == user_id)
    return list(self.db.execute(stmt).scalars().all())
```



El Viaje de la Response



```
| } |
```

Resumen de Transformaciones

Paso	Input	Output
Cliente → Router	JSON	OrderCreate (DTO)
Router → Service	OrderCreate	Order (Entity)
Service → Repository	Order	Order (con ID)
Repository → Service	Order	Order
Service → Router	Order	Order
Router → Cliente	Order → OrderResponse	JSON

Checklist

- ☐ Entiendo el flujo completo de request a response
- ☐ Sé qué transformaciones ocurren en cada capa
- ☐ Puedo identificar responsabilidades por capa
- ☐ Entiendo cómo fluyen los datos entre capas